AUGUST 28 2026

Institutional Considerations for Post-Quantum Migration

Yehuda Lindell Head of Cryptography at Coinbase

coinbase

Customer Concern

The Quantum Threat

- There is a lot of noise out there and customers can't distinguish the signal from the noise
- The question of **when** is indeed unclear, but the concern is real
- The question isn't when it is likely, but rather **when it's possible**

Quantum and Bitcoin Investment

- All investments have risk, but what happens when you add existential threat to existing volatility?
- Customers are expressing growing concern (very high percentages of UHNW and institutional customers in a high percentage of conversations)
- We know of multiple cases of not investing due to quantum

The main thing customers want to know is that there will be a solution and on time – the biggest concern is inaction

Custodial / Institutional Challenge

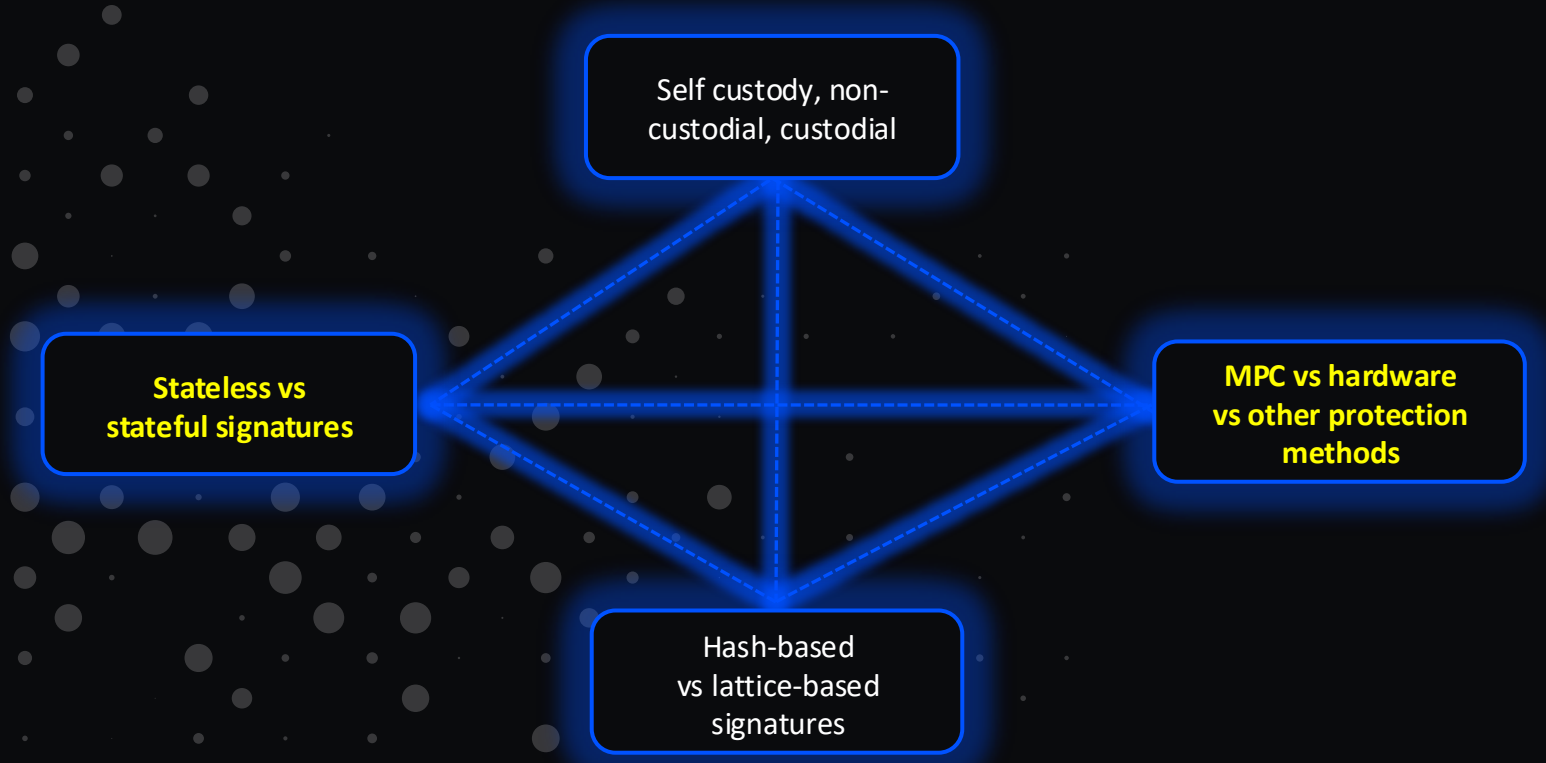
Scale:

- Multiple blockchains with different signing schemes
- System changes need to be disclosed to regulators and some customers
- Asset migration – massive transfers need to be planned and are an operational and security challenge as well

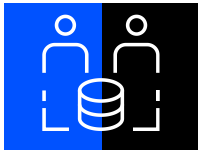
Stakes:

- Mistakes at this scale can be disastrous, to the organization and to the industry (threat of loss of trust)

Issues to be considered



Ownership Models



Self custody / user wallets:

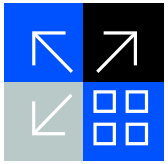
- Millions of independent key owners each need to migrate
- Wallets need to support new algorithms
- Some users will never migrate, causing additional problems

Custody:

- Custodian holds all funds and can migrate for all users
- Custodian's KMS needs to support all new algorithms on all relevant blockchains
- Concentrated risk

Non-custodial:

- This can refer to a situation that is neither self custody or custody
- The key can be split between the institution and user, with MPC



MPC / Threshold Signing

Keys are split and never brought together during signing

Uses:

- **Custodians:** improve security by preventing any single point of failure
  - Keys split over multiple machines and environments
- **Non-custody:** no ownership by institution (not custody) but breach of the user doesn't enable key theft

The challenge: Not every signature scheme has an efficient MPC protocol

- Lattice-based signatures should be fine
- Hash-based signatures are a completely different story

Saying **"use multisig instead"** is not a good answer in many cases, especially for **institutional**

SLH-DSA – A Case Study

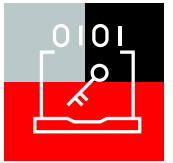
Parameter sets and number of hash invocations:

- **SLH-DSA-SHA2-128f (sig size 16.7KB):** over 100,000 hash ops to sign
- **SLH-DSA-SHA2-128s (sig size 7.7KB):** over 2,000,000 hash ops to sign

Consider an MPC protocol with just 32 bytes of communication per AND gate (we don't know of any such protocol)

- **SHA2 has ~22,000 AND gates per invocation**
- **SLH-DSA-SHA2-128f:** communication >> 66 GB
  - At 100Mbps, this would take about 1.5 hours just to upload *(in reality, the best protocols we know are more than double this)*
  - This isn't inconceivable in internal custody cases *(can parallelize),* **but it's unclear that 128-bits of security suffices**
- **SLH-DSA-SHA2-128s:** communication >> 1.3 TB
  - At 100Mbps, this would take over a full day just to upload

Ramifications of MPC Hardness



Non-custodial solutions that rely on splitting the key between the user and the provider have a major problem

Custodians who utilize MPC for internal security need to search for other models that may not be as robust

The bottom line:

The “safest” signature scheme may be one that is not thresholdizable, weakening other parts of the ecosystem

Potential alternative approaches:

- MPC-friendly MPC-in-the-Head ZK-based or other signatures
- Demonstrated to be competitive in size, but not standardized (and actively under research now)

Stateless versus Stateful Signatures

Stateless Signatures

- Each signature is a function of the key and the message only
- Back up the key once, sign forever, from any device
- The standard workflow for almost every signature scheme in production today

Stateful Signatures (XMSS, LMS, leanXMSS)

- The signer must track which one-time keys have been used and never reuse a state
- Signatures are smaller and faster

Critical:
Reusing a state can be catastrophic, allowing immediate forgery

Stateful Signatures Another Approach

Stateful signatures (part of the SHRINCS recommendation) **can** be much smaller and much faster

- Faster = fewer hash operations = more MPC friendly
- One parameter setting for SHRINCS requires just 938 hash operations to sign

So isn't this is a good solution overall?

- Much smaller
- More amenable to MPC
- Has stateless fallback if needed

Using Stateful Signatures – Single Wallet Case

Secure process for signing

Read current state → sign → save the updated state → output the signature

Challenges

- The above is secure, but the assumption that the state can be verifiably updated on an ongoing basis is harder
  - Error in writing (write caches, rewind of the DB by someone not realizing that the DB also holds this critical state, etc.)
- An error doesn't result in a hassle, but in publicly compromising the key

Even in this simple case, a routine operational mistake (a bad restore or race condition) can destroy security while everything looks normal

Using Stateful Signatures – Institutional System

Today:

Multiple server instances operate independently to process signatures

Stateful:

Architecture needs to completely change

1. Cannot utilize independent instances in different zones for high availability and fast response
2. Exclusive read & write lock on a key and its state during use
3. Transactions requiring multiple signatures – special processing
4. Key-state storage requires write-commit at every operation
5. Regular DB usage that enables restore from backup cannot be used
   - Need to separate from regular storage (today, can encrypt and store on a regular DB)

Stateful Signature Error

Lost state is “not” a problem

- With SHRINCS can just reset with the stateless scheme

Mistaken state reuse is a disaster

- You don’t necessarily know that it has happened
  - You can’t rely on blockchain scanning – not all sent signatures make it to the blockchain
- Mistakes can be public, and result in complete asset theft

You don’t need many errors to erode trust

Mitigation (Dan Boneh): use 2-of-2 signatures with ECDSA when using stateful

No Single Good Choice

1:

Hash-based

- Most conservative assumption-wise, but not MPC friendly
- Stateful – operational resulting in security concerns

2:

Lattice-based

- Not as conservative, but much more MPC friendly

- There can and will be others **(MPC-in-the-Head hash-based for one example)** but do we have time to wait?

Protocol Optionality With a Kill Switch

Example solution – **support multiple signing schemes:**

- ECDSA/Schnorr, stateful hash, stateless hash, lattice based
- One address with 1-of-N for the different types
  - All keys hashed until used so vulnerabilities on unused types don't affect security
  - The blockchain blocks a type if a vulnerability is discovered or close
- Institutional autonomy – allow different orgs with different tradeoffs to choose what they want to use

Food for Thought

Solution Dilemma

- There is no one good solution for all (blockchain restraints, individual wallets, custodians, institutional non-custodial)
- What will the impact of a solution that is good for X but bad for Y?

Solution Choice

- How much flexibility is viable for the initial solution and going forward?
- Is Bitcoin looking for a single option, or will this be a process of adding and changing?

Timelines and Effort

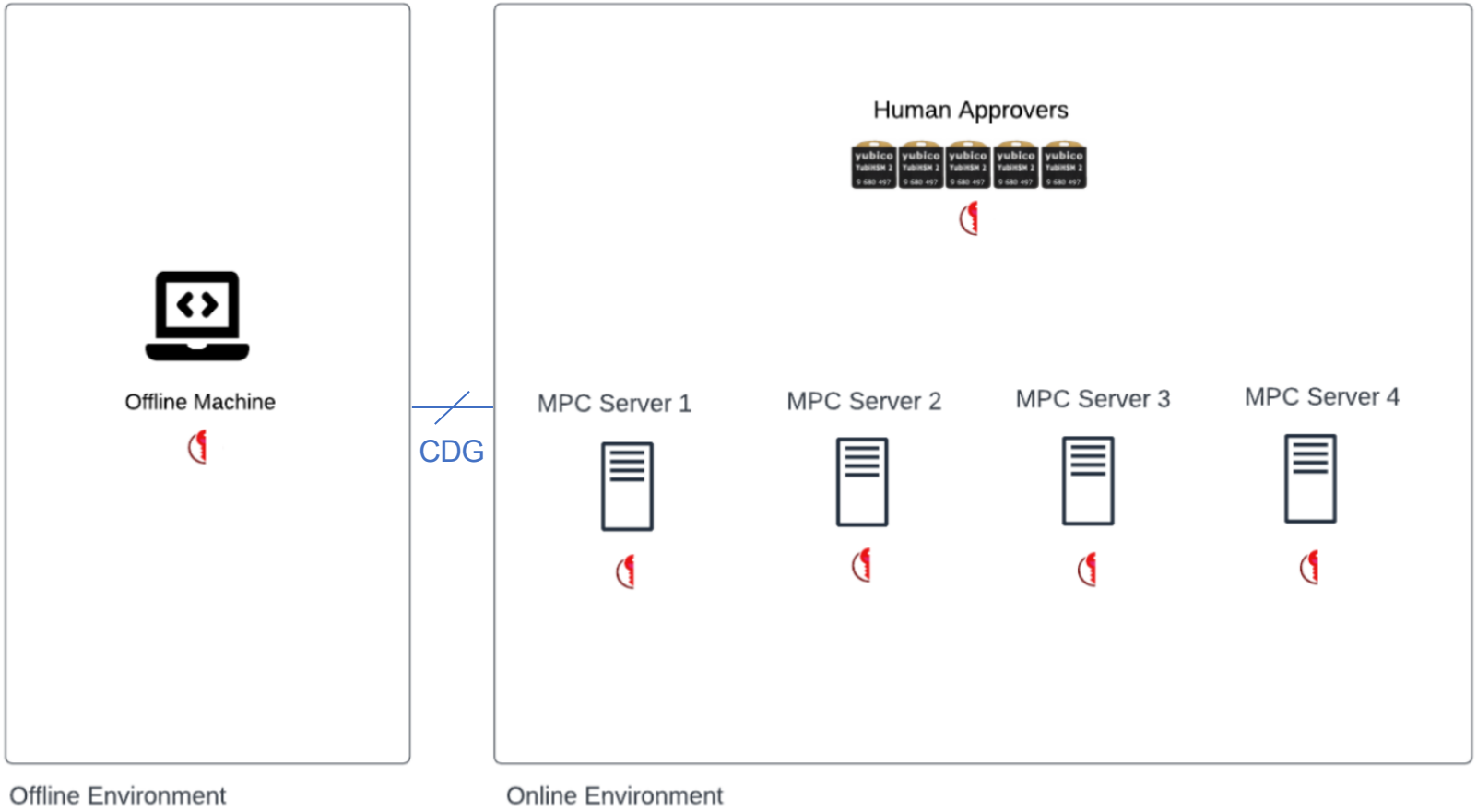
- Do we have time to search for better options? Is this a good use of resources?

  We can spend time developing MPC-friendly hash-based signatures or developing protocols for best existing candidates.

Institutional Considerations for Post-Quantum Migration

Thank You

coinbase


Human Approvers
Offline Machine
CDG
MPC Server 1
MPC Server 2
MPC Server 3
MPC Server 4
Offline Environment
Online Environment

Standard Algorithms and Hardware

Standard HSMs will have ML-DSA, SLH-DSA and other NIST standardized algorithms

The Good

- The use of standard vetted hardware is typically better

The Bad

- The tradeoffs for these standards are not necessarily the best for blockchain